

Project work

WP1 – Modello del Sistema

L'obiettivo del progetto è la progettazione di un protocollo di voto elettronico sicuro per un referendum binario, nel quale ciascun elettore può esprimere una sola preferenza tra due possibili alternative: "SÌ" e "NO". La scelta di un sistema binario permette di concentrare l'attenzione sui principali problemi di sicurezza del voto elettronico senza introdurre ulteriori complessità legate alla gestione di più candidati o preferenze ordinate. Nonostante la semplicità dello scenario, il problema rimane di grande interesse crittografico poiché il sistema deve garantire simultaneamente autenticazione degli elettori, segretezza del voto, integrità del risultato e verificabilità pubblica. Il voto elettronico rappresenta oggi una possibile evoluzione dei sistemi elettorali tradizionali grazie alla capacità di ridurre i tempi di scrutinio, migliorare l'accessibilità e diminuire i costi organizzativi. Tuttavia, la sua adozione introduce criticità importanti, tra cui:

- violazione della segretezza del voto;
- manipolazione dei risultati;
- impossibilità di verificare la correttezza del conteggio;
- attacchi alla disponibilità del sistema;
- rischio di coercizione o acquisto del voto.

Si assume che gli attaccanti possano essere sia esterni che interni al sistema, e possano agire in modo passivo (eavesdropping) o attivo (modifica, iniezione o replay di messaggi). In particolare, il canale di comunicazione può essere intercettato o manipolato, rendendo necessari meccanismi crittografici per garantire confidenzialità, integrità e autenticazione. Per questo motivo il protocollo proposto mira a raggiungere un compromesso ragionevole tra efficienza, trasparenza, segretezza e sicurezza, utilizzando esclusivamente strumenti crittografici coerenti con quelli affrontati durante il corso. Il protocollo segue i principali principi di progettazione della sicurezza informatica, tra cui il principio di least privilege, la mediazione completa degli accessi e il principio di open design, secondo cui la sicurezza non deve dipendere dalla segretezza del sistema ma esclusivamente delle chiavi crittografiche.

La scelta del referendum binario consente inoltre di:

- ridurre la complessità crittografica del protocollo;
- semplificare la rappresentazione delle schede di voto;
- facilitare le verifiche di correttezza del conteggio;
- contenere i costi computazionali e le latenze.

1.1 Attori del sistema

Per definire chiaramente i confini di fiducia tra le diverse componenti, il sistema è organizzato in più moduli separati, ciascuno con responsabilità specifiche.

1.1.1 Elettori

Gli elettori sono gli utenti autorizzati a partecipare alla votazione. Ogni elettore possiede credenziali di autenticazione valide e utilizza un software client per generare e inviare la propria scheda.

Project work

Il client produce localmente il voto, lo cifra tramite la chiave pubblica dell'Autorità Elettorale e lo invia al sistema insieme a un token anonimo di autorizzazione. L'elettore riceve una ricevuta o identificatore univoco che gli permetterà successivamente di verificare la presenza del proprio voto nella bacheca pubblica.

Gli elettori devono poter:

- votare una sola volta;
- mantenere segreta la propria preferenza;
- verificare autonomamente la registrazione del proprio voto.

Gli elettori possono essere sia benevoli sia malevoli, e il protocollo deve essere progettato assumendo che alcuni possano tentare di abusare del sistema.

1.1.2 Autorità di Registrazione / Autenticazione

L'Autorità di Registrazione (o Autorità di Autenticazione) è responsabile dell'identificazione degli elettori e della verifica del loro diritto di voto. Questa entità è l'unica a conoscere l'identità reale degli utenti, ma non deve essere in grado di conoscere il contenuto dei voti espressi.

I suoi compiti principali sono:

- autenticare gli elettori tramite credenziali verificabili;
- controllare che ogni elettore abbia diritto al voto;
- impedire votazioni multiple;
- rilasciare un token anonimo utilizzabile una sola volta.

L'autorità mantiene traccia dei token già emessi così da evitare che uno stesso elettore possa ricevere più autorizzazioni.

1.1.3 Autorità Elettorale / Server di voto

L'Autorità Elettorale si occupa della raccolta dei voti, della loro conservazione e dello scrutinio finale. Questa entità riceve esclusivamente schede cifrate e token anonimi, senza conoscere direttamente l'identità dell'elettore che le ha inviate.

L'Autorità Elettorale:

- genera una coppia di chiavi crittografiche asimmetriche;
- pubblica la chiave pubblica necessaria alla cifratura dei voti;
- conserva segreta la chiave privata utilizzata solo al termine della votazione;
- verifica la validità dei token ricevuti;
- registra i voti validi;
- esegue il conteggio finale;
- pubblica le informazioni necessarie alla verificabilità del sistema.

Il server di voto non deve essere considerato completamente fidato. Il protocollo deve quindi limitare la sua capacità di alterare voti o compromettere la segretezza del sistema. Si assume che l'Autorità di Registrazione e l'Autorità Elettorale non colludano tra loro.

Project work

1.1.4 Bacheca Pubblica

La Bacheca Pubblica è un archivio accessibile in sola lettura che contiene:

- i voti cifrati;
- gli identificatori delle schede;
- il risultato finale;
- le eventuali prove necessarie alla verifica pubblica.

La bacheca rappresenta il principale strumento di trasparenza del sistema. Poiché i dati pubblicati sono accessibili a tutti, ogni elettore può controllare la presenza del proprio voto mentre osservatori esterni possono verificare la correttezza del conteggio.

La bacheca può essere implementata come:

- database pubblico consultabile;
- registro append-only;
- file firmato digitalmente.

1.1.5 Osservatori pubblici

Gli osservatori pubblici sono soggetti esterni che non partecipano direttamente alla votazione ma verificano la correttezza del risultato finale.

Essi devono poter:

- controllare la coerenza delle informazioni pubblicate;
- verificare il numero delle schede conteggiate;
- controllare che il risultato finale corrisponda ai voti registrati.

1.2 Funzionalità del sistema

Il protocollo segue una struttura suddivisa in quattro fasi principali.

Fase 1 – Registrazione e autenticazione

L'elettore effettua l'accesso tramite credenziali valide.

L'Autorità di Registrazione:

- verifica l'identità dell'utente;
- controlla che non abbia già ottenuto un token;
- genera un token anonimo monouso.

Il token certifica il diritto di voto ma non contiene informazioni sulla preferenza espressa.

Fase 2 – Espressione del voto

L'elettore utilizza il token ricevuto per inviare la scheda cifrata all'Autorità Elettorale.

Il voto:

- viene generato localmente dal client;
- viene cifrato mediante la chiave pubblica dell'Autorità Elettorale;

Project work

- non contiene informazioni identificative;
- viene associato soltanto a un identificatore casuale.

Il sistema deve garantire che il voto non possa essere alterato o sostituito durante la trasmissione.

Fase 3 – Pubblicazione e scrutinio

Al termine della votazione l'Autorità Elettorale:

- decrittifica le schede valide;
- calcola il risultato finale;
- pubblica i voti cifrati e gli identificatori sulla bacheca pubblica;
- pubblica il risultato finale e le eventuali prove di correttezza.

Fase 4 – Verifica

Ogni elettore può verificare che il proprio voto sia stato registrato confrontando la ricevuta ricevuta durante la votazione con gli identificatori presenti nella bacheca pubblica.

Gli osservatori esterni possono invece controllare che:

- il numero delle schede conteggiate coincida con quelle pubblicate;
- il risultato finale sia coerente con l'insieme delle schede scrutinabili.

1.3 Threat Model

Per valutare correttamente la sicurezza del protocollo è necessario definire un modello di minaccia realistico, considerando i possibili avversari e le loro capacità.

1.3.1 Gestore disonesto del server

L'Autorità Elettorale potrebbe tentare di:

- modificare voti validi;
- eliminare schede;
- aggiungere voti arbitrari;
- alterare il conteggio finale;
- correlare identità e preferenze.

Il protocollo deve mitigare tali rischi tramite la separazione delle responsabilità, la pubblicazione delle schede cifrate e la verificabilità pubblica.

1.3.2 Elettore malevolo

Un elettore fraudolento potrebbe cercare di:

- votare più volte;
- utilizzare token rubati;
- impersonare altri utenti;
- ripudiare il proprio voto;
- dimostrare a terzi la preferenza espressa;

Project work

- modificare il client di voto.

Il sistema deve garantire che ciascun token sia utilizzabile una sola volta e che eventuali comportamenti anomali risultino rilevabili.

1.3.3 Avversario esterno

Un osservatore esterno potrebbe intercettare le comunicazioni di rete nel tentativo di:

- leggere il contenuto dei messaggi;
- modificare le comunicazioni;
- ritrasmettere pacchetti intercettati;
- compromettere la disponibilità del sistema tramite attacchi DoS.

Si assume tuttavia che tale avversario non possieda le chiavi private delle autorità coinvolte.

1.3.4 Collusione tra autorità

Uno degli scenari più critici è rappresentato dalla collusione tra Autorità di Registrazione e Autorità Elettorale.

Se entrambe condividessero tutte le informazioni in loro possesso, sarebbe possibile ricostruire l'associazione tra identità reale dell'elettore e voto espresso.

Il protocollo non è in grado di eliminare completamente questo rischio ma assume, come ipotesi di trust, che le due entità siano amministrate da soggetti distinti e non colludano.

Questa rappresenta la principale limitazione strutturale del sistema.

1.4 Proprietà di sicurezza desiderate

Il protocollo deve garantire un insieme di proprietà fondamentali per assicurare la correttezza e l'affidabilità del processo elettorale.

Autenticità

Solo gli elettori autorizzati devono poter partecipare alla votazione.

Il sistema deve verificare l'identità dell'utente prima del rilascio del token di voto.

Unicità

Ogni elettore deve poter esprimere al massimo una preferenza valida.

Il sistema deve impedire:

- doppi voti;
- riutilizzo dei token;
- replay dei messaggi.

Segretezza del voto

Nessuna singola entità deve essere in grado di associare direttamente un voto all'identità dell'elettore.

Project work

L'Autorità di Registrazione conosce l'identità ma non il voto, mentre l'Autorità Elettorale conosce il voto cifrato ma non l'identità del votante.

La segretezza è garantita sotto l'ipotesi di non collusione.

Integrità

I voti non devono poter essere modificati, eliminati o sostituiti senza che l'alterazione sia rilevabile.

La pubblicazione delle schede cifrate nella bacheca pubblica permette di individuare eventuali manomissioni o omissioni.

Verificabilità individuale

Ogni elettore deve poter verificare autonomamente che il proprio voto sia stato incluso nella raccolta finale.

Questa verifica viene effettuata tramite la ricevuta ottenuta durante la procedura di voto.

Verificabilità universale

Chiunque deve poter verificare pubblicamente la correttezza del risultato finale.

Gli osservatori devono poter controllare:

- il numero totale delle schede;
- la coerenza del conteggio;
- la corrispondenza tra voti pubblicati e risultato finale.

Trasparenza

Il protocollo deve minimizzare la fiducia richiesta nelle singole componenti.

Ogni assunzione di trust deve essere:

- esplicita;
- motivata;
- analizzata criticamente.

Efficienza

Il sistema deve essere eseguibile anche su dispositivi ordinari senza costi computazionali eccessivi.

Nel caso del referendum binario, la ridotta dimensione delle schede e la semplicità delle operazioni crittografiche permettono un'implementazione concretamente sostenibile.

1.5 Conclusioni

Il modello definito descrive un sistema di voto elettronico basato sulla separazione tra autenticazione e scrutinio, sull'utilizzo della crittografia asimmetrica e sulla presenza di una bacheca pubblica verificabile.

Project work

Il protocollo è progettato per garantire autenticità, unicità, segretezza, integrità e verificabilità del processo elettorale, assumendo che l'Autorità di Registrazione e l'Autorità Elettorale siano entità separate e non colludano.

Questo modello costituirà la base per la progettazione dettagliata del protocollo che verrà sviluppata nel WP2.

Project work

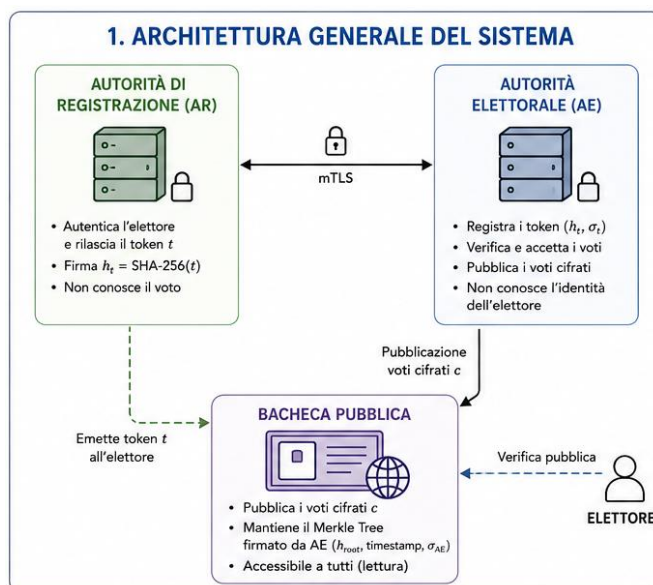
WP2 – Soluzione

Il protocollo proposto è strutturato in cinque fasi sequenziali: (i) distribuzione delle chiavi, (ii) registrazione e autenticazione dell'elettore, (iii) espressione e trasmissione del voto cifrato, (iv) scrutinio finale, (v) verifica individuale e universale. L'obiettivo è raggiungere un ragionevole compromesso tra segretezza, integrità e verificabilità, impiegando esclusivamente gli strumenti trattati nel corso.

Il sistema prevede tre componenti logicamente separati:

- **Autorità di Registrazione (AR):** autentica gli elettori, verifica il diritto di voto e rilascia token monouso anonimi. Conosce l'identità di ogni elettore, ma non il suo voto.
- **Autorità Elettorale (AE):** raccoglie le schede cifrate, ne verifica la validità tramite i token, mantiene la bacheca pubblica ed esegue lo scrutinio. Riceve voti cifrati anonimi, ma non sa a chi appartengono.
- **Bacheca Pubblica (BP):** registro append-only pubblicamente leggibile, gestito da un soggetto terzo o implementato su un registro verificabile. Contiene tutte le schede cifrate, gli identificatori pubblici e il risultato finale.

Assunzione di fiducia fondamentale: AR e AE non colludono. Questa è l'assunzione critica su cui poggia lo pseudo-anonimato del sistema. Se le due autorità collaborassero, potrebbero in linea di principio correlare identità e voto. Il rischio residuo è analizzato in WP3. Si assume inoltre che entrambe le autorità siano dotate di certificati digitali validi rilasciati da una CA riconosciuta, e che la Bacheca Pubblica garantisca append-only verificabile tramite firma dell'AE sulla radice del Merkle Tree.



2.1 Generazione e distribuzione delle chiavi

2.1.1 Chiavi dell'Autorità Elettorale

Prima dell'apertura delle urne, l'AE esegue l'algoritmo Gen(1ⁿ) RSA per generare la propria coppia di chiavi (pkAE, skAE). Il parametro n determina la lunghezza del modulo; nella

Project work

realizzazione si utilizzano moduli di almeno 2048 bit (consigliati 3072 bit per margine di sicurezza a lungo termine). L'algoritmo di generazione procede come segue: si selezionano due primi grandi p e q ; si calcola $N = pq$ e $\phi(N) = (p-1)(q-1)$; si sceglie e con $\gcd(e, \phi(N)) = 1$; si ricava $d = e^{-1} \bmod \phi(N)$. La chiave pubblica è $pkAE = (N, e)$ e quella privata è $skAE = (N, d)$. Pubblicazione di $pkAE$. La chiave pubblica viene distribuita agli elettori tramite un canale autenticato:

- l'AR recupera $pkAE$ dall'AE attraverso il canale TLS mutualmente autenticato tra le due autorità (descritto in §2.2.4);
- l'AR firma $pkAE$ con la propria chiave privata $skAR$, producendo $\sigma_{pk} = \text{Sign}_{skAR}(pkAE)$;
- la coppia $(pkAE, \sigma_{pk})$ è pubblicata nella Bacheca Pubblica prima dell'apertura delle urne;
- ogni elettore verifica $\text{Vrfy}_{pkAR}(pkAE, \sigma_{pk}) = 1$ prima di usare $pkAE$ per cifrare il voto.

2.1.2 Chiavi dell'Autorità di Registrazione

L'AR possiede una coppia $(pkAR, skAR)$ con certificato digitale rilasciato da una CA radice riconosciuta. $pkAR$ è pubblica e nota a tutti i partecipanti. $skAR$ è usata per: firmare il certificato di $pkAE$, autenticare i token nel registro condiviso con l'AE e autenticare le comunicazioni TLS lato AR.

2.1.3 Schema riassuntivo delle chiavi

Entità	Chiave pubblica	Chiave privata	Uso
AR	$pkAR$	$skAR$	Firma token, TLS, certificazione $pkAE$
AE	$pkAE$	$skAE$	Cifratura voti, firma radice Merkle
AE	$pkAE\text{-cert}$	$skAE\text{-cert}$	Autentica la pubblicazione di $skAE$ a urne chiuse

Nota: Il certificato di $pkAE\text{-cert}$ è rilasciato dalla stessa CA radice che certifica $pkAR$. Questo introduce un'assunzione di fiducia verso tale CA, analoga a quella già dichiarata per $pkAR$. Il rischio residuo è lo stesso: una CA compromessa potrebbe certificare una chiave falsa. Poiché entrambe le autorità (AR e AE) dipendono dalla stessa CA, questa assunzione è già implicita nel modello e non aggiunge fiducia ulteriore.

2.1.4 Canale sicuro tra AR e AE

AR e AE mantengono un canale TLS mutualmente autenticato (mTLS), stabilito nella fase di setup prima dell'apertura delle urne. Entrambe le parti si autenticano tramite i rispettivi certificati digitali: l'AR con il certificato di $pkAR$ e l'AE con il certificato di $pkAE\text{-cert}$, entrambi rilasciati dalla CA radice comune. Il canale viene stabilito una sola volta e rimane attivo per tutta la durata dell'elezione; ogni messaggio trasmesso su di esso è protetto dalla sessione TLS attiva (confidenzialità e integrità). Attraverso questo canale, l'AR trasmette all'AE gli hash dei token ht firmati, e l'AE comunica all'AR eventuali anomalie nel registro delle autorizzazioni.

Project work

2.2 Registrazione e autenticazione dell'elettore

2.2.1 Apertura del canale sicuro verso l'AR

L'elettore apre una sessione TLS 1.3 verso l'AR. Il protocollo TLS autentica il server tramite il certificato digitale dell'AR e, attraverso uno scambio Diffie-Hellman effimero (DHE), le due parti concordano una chiave di sessione master ks:

$$h_A = g^x \bmod p \text{ [elettore} \rightarrow \text{AR]}$$

$$h_B = g^y \bmod p \text{ [AR} \rightarrow \text{elettore]}$$

$$k_s = g^{xy} \bmod p$$

Derivazione delle sottochiavi. Da k_s vengono derivate due chiavi indipendenti tramite HKDF (funzione di derivazione basata su HMAC):

$$k_E = \text{HKDF}(k_s, \text{"enc"})$$

$$k_M = \text{HKDF}(k_s, \text{"mac"})$$

L'indipendenza crittografica di k_E e k_M è garantita dall'uso di etichette distinte nella derivazione. Derivare entrambe le chiavi direttamente da k_s senza separazione (ad es. usando k_s sia per la cifratura sia per il MAC) invaliderebbe le garanzie di sicurezza dello schema Encrypt-then-Authenticate. Le chiavi k_E e k_M sono poi usate in modalità Encrypt-then-Authenticate: la cifratura simmetrica CPA-sicura (AES-CTR) è applicata prima, poi un MAC con chiave indipendente k_M autentica il ciphertext. Nonce e timestamp sono inclusi in ogni messaggio per prevenire attacchi di replay. La struttura del messaggio protetto è la seguente: si calcola $c \leftarrow \text{AES-CTR}_{k_E}(m)$ e poi $t \leftarrow \text{Mac}_{k_M}(c)$, trasmettendo la coppia $\langle c, t \rangle$. Il destinatario verifica prima il tag sul ciphertext ($\text{Vrfy}_{k_M}(c, t) = 1$) e solo se valido decifra ($m \leftarrow \text{AES-CTR}_{k_E}(c)$). Questo è lo schema Encrypt-then-Authenticate (Construction 5.6 degli appunti), che garantisce CCA-security con chiavi indipendenti k_E e k_M .

2.2.2 Verifica dell'identità e rilascio del token anonimo

Una volta aperto il canale sicuro, l'elettore presenta le proprie credenziali (nome utente e password, o certificato personale). L'AR verifica che:

- l'elettore figuri nella lista degli aventi diritto (verificato sulla base del registro elettorale);
- non abbia già ottenuto un token attivo in questa elezione (controllo anti-doppio-voto lato AR).

Superata l'autenticazione, l'AR deve rilasciare all'elettore una prova di autorizzazione che non consenta all'AE di risalire all'identità originale. Il meccanismo scelto è il seguente:

- Generazione del token: l'AR genera una stringa casuale $t \leftarrow \mathcal{R} \{0,1\}^{256}$ e calcola il suo hash $ht = \text{SHA-256}(t)$.
- Comunicazione all'AE: l'AR trasmette ht all'AE — sul canale mTLS descritto in §2.2.4 — corredato dalla firma $\sigma_t = \text{Sign}_{skAR}(ht)$ prodotta con RSA-FDH. L'AE

Project work

verifica immediatamente $\text{Vrfy_pkAR}(ht, \sigma t) = 1$: se la verifica fallisce, il token è rifiutato e non inserito nel registro (cfr. §2.10, caso "Firma σt non valida"). Se la verifica ha esito positivo, l'AE inserisce ht nel proprio registro delle autorizzazioni valide, accessibile soltanto a se stessa.

- Consegna all'elettore: il valore t è consegnato in chiaro all'elettore sul canale TLS.

Questo meccanismo garantisce:

- Separazione delle conoscenze: l'AR conosce l'identità dell'elettore e t , ma non il voto; l'AE riceve ht firmato e successivamente il voto cifrato, ma non sa a quale identità corrisponda.
- Autenticità del token: la firma σt garantisce che solo l'AR certificata possa aver emesso quel token, impedendone la falsificazione da parte dell'elettore o di terzi.
- Monouso: non appena ht viene presentato all'AE durante la votazione, è rimosso dal registro. Tentativi di riuso sono rigettati.

Limite del meccanismo: L'AR conosce la corrispondenza (identità, t). Se l'elettore trasmette il voto subito dopo aver ricevuto t , un avversario che controlli il timing delle connessioni verso l'AE potrebbe tentare una correlazione temporale. Questo rischio è mitigato dall'architettura (le due sessioni TLS sono indipendenti) ma non eliminato. Un'estensione futura con firma cieca (blind signature) eliminerebbe questo legame anche dal punto di vista crittografico, ma richiede strumenti non trattati nel corso.

2.3 Espressione e trasmissione del voto

2.3.1 Struttura della scheda cifrata

Il voto è binario: l'elettore sceglie $v = 0$ (NO) oppure $v = 1$ (SÌ). Prima di cifrare, il client genera un identificatore casuale $id \leftarrow R \{0,1\}^{128}$: questo valore fungerà da *ricevuta personale* e permetterà all'elettore di ritrovare la propria scheda nella bacheca senza esporre informazioni identificative.

Formazione del plaintext. Il client forma:

$$m = v \parallel id \quad (|m| = 129 \text{ bit})$$

Il plaintext m è cifrato con RSA-OAEP (schema trattato nel corso) usando pk_{AE} :

$$c = \text{OAEP-Enc}(pk_{AE}, m)$$

Durante lo scrutinio, l'AE (e qualsiasi verificatore esterno) controlla che $v_i \in \{0, 1\}$ dopo la decifrazione. Le schede per cui il campo v assume un valore diverso sono scartate dal conteggio e segnalate come non valide nella bacheca.

RSA-OAEP introduce un padding pseudocasuale che garantisce il non-determinismo del ciphertext: due cifrature dello stesso voto producono ciphertext completamente diversi, impedendo a un osservatore — inclusa l'AE stessa durante la votazione — di dedurre la distribuzione dei voti per confronto diretto o conteggio delle occorrenze. Lo schema è dimostrabilmente IND-CCA2 sicuro nel Random Oracle Model (ROM), sotto l'assunzione che il problema RSA sia computazionalmente difficile.

Project work

2.3.2 Autenticazione della scheda e struttura del messaggio

Per garantire l'integrità della scheda durante la trasmissione, il client calcola una firma sul ciphertext. Poiché l'AE non conosce l'identità dell'elettore, non è possibile usare una chiave pubblica certificata; si utilizza invece una coppia di chiavi RSA effimere (pk_e , sk_e) generata localmente per questa sola operazione, con lo stesso algoritmo $\text{Gen}(1^n)$ descritto in §2.2.1 e modulo di almeno 2048 bit. La chiave effimera è usa-e-getta: viene generata appena prima dell'invio e conservata localmente dall'elettore fino alla chiusura delle urne, per consentire l'eventuale firma di una richiesta di revoca (§2.8). Alla chiusura delle urne, o dopo conferma che la revoca non è necessaria, sk_e viene eliminata dal dispositivo dell'elettore. La firma segue lo schema RSA-FDH (coerentemente con §2.10). In RSA-FDH (Construction 13.6), l'algoritmo Sign applica internamente la funzione H al messaggio prima di esponenziare: $\sigma = H(m)^d \bmod N$. Il messaggio da firmare è $m = c \parallel t$, con $H = \text{SHA-256}$ già incorporata nello schema; SHA-256 non va applicata esternamente prima di invocare Sign:

$$\sigma_v = \text{Sign}_{sk_e}(c \parallel t) \text{ [RSA-FDH, con } H = \text{SHA-256} \text{ incorporata]}$$

L'inclusione del token t nel messaggio firmato vincola la firma alla specifica scheda e alla specifica autorizzazione, impedendo attacchi di riuso di una firma su un ciphertext diverso. La dimensione di pk_e trasmessa all'AE è pari alla dimensione del modulo RSA (≈ 256 byte per moduli a 2048 bit). Scopo della chiave effimera. La firma con chiave effimera assolve due funzioni: garantisce l'integrità della scheda in transito e consente all'AE di verificare che mittente e token siano coerenti. La chiave effimera non certifica l'identità reale dell'elettore, il che è intenzionale: l'anonimato è preservato proprio perché pk_e non è associabile ad alcuna identità reale.

Messaggio trasmesso all'AE:

$$M_{\text{voto}} = (c, t, pk_e, \sigma_v)$$

2.3.3 Invio della scheda all'Autorità Elettorale

Il client apre una nuova sessione TLS verso l'AE, separata e indipendente da quella verso l'AR. Questa separazione limita la possibilità di correlazione tra i due canali.

Verifiche dell'AE alla ricezione di M_{voto} :

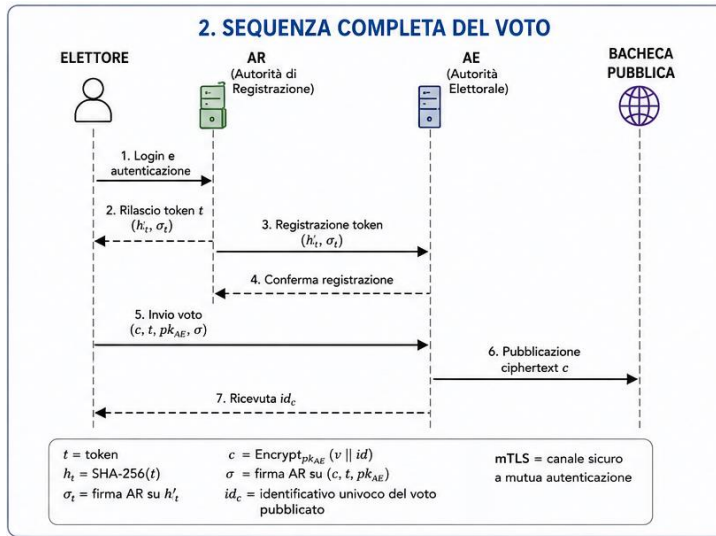
1. Presenza nel registro: l'AE calcola $ht' = \text{SHA-256}(t)$ e verifica che ht' sia presente nel proprio registro delle autorizzazioni valide. Quel registro contiene esclusivamente hash di token già autenticati tramite σ_t al momento della trasmissione $AR \rightarrow AE$ (§2.3.2); la verifica crittografica della firma è quindi già stata eseguita in quella fase.
2. Integrità della scheda: l'AE verifica $\text{Vrfy}_{pk_e}(c \parallel t, \sigma_v) = 1$ — il messaggio passato a Vrfy è $c \parallel t$ come stringa di bit; SHA-256 è applicata internamente da RSA-FDH, non esternamente dal verificatore.
3. Unicità: l'AE controlla che il token non sia già stato utilizzato per registrare un'altra scheda in questa elezione.

Azioni dell'AE se tutte le verifiche hanno esito positivo:

- registra c tra le schede valide;

Project work

- cancella ht dal registro delle autorizzazioni (token monouso);
- calcola $idc = \text{SHA-256}(c)$ e lo restituisce all'elettore come conferma;
- pubblica nella Bacheca Pubblica la coppia (c, idc) .



2.4 Bacheca Pubblica e integrità dei dati

2.4.1 Struttura del Merkle Tree

La Bacheca Pubblica è organizzata come un Merkle Tree append-only. Ogni scheda cifrata ci è inserita come foglia dell'albero:

- Hash di foglia: $h(ci) = \text{SHA-256}(ci)$
- Nodi interni: $h(hl \parallel hr) = \text{SHA-256}(hl \parallel hr)$, dove hl e hr sono gli hash dei figli sinistro e destro.
- Radice: la radice hroot riassume l'intera struttura.

Ad ogni inserimento di una nuova scheda, l'AE ricalcola i nodi del percorso dalla foglia alla radice e aggiorna e ri-firma hroot con RSA-FDH:

$$\sigma_{\text{root}} = \text{Sign}_{sk_{AE}}(\text{hroot} \parallel \text{timestamp})$$

Il messaggio firmato è la concatenazione $\text{hroot} \parallel \text{timestamp}$ (canonicalizzazione fissa).

L'inclusione del timestamp nella firma impedisce la sostituzione di una versione più vecchia dell'albero. La firma è pubblicata nella Bacheca Pubblica insieme alla radice, consentendo a chiunque di verificarne l'autenticità con pk_{AE} tramite:

$$\text{Vrfy}_{pk_{AE}}(\text{hroot} \parallel \text{timestamp}, \sigma_{\text{root}}) = 1$$

Ogni firma σ_{root} costituisce uno snapshot verificabile dello stato della bacheca al momento dell'inserimento. Le firme precedenti non vengono invalidate: rimangono in bacheca e consentono di ricostruire la storia completa degli inserimenti. Un verificatore può quindi accertare non solo lo stato finale, ma anche ogni stato intermedio della bacheca.

Project work

2.4.2 Merkle Proof e verifica di appartenenza

Per dimostrare che ci è presente in bacheca, è sufficiente la Merkle Proof associata: la sequenza di hash $\pi = (h_1, \dots, h_j)$ che permette di ricostruire hroot a partire da $h(c_i)$. Il costo della verifica è $O(\log n)$, dove n è il numero di schede. La firma σ_{root} vincola l'intera lista: nessuna scheda può essere eliminata, modificata o riordinata senza invalidare la firma e rendere evidente la manomissione.

2.4.3 Numero pubblico di token emessi

Al termine della votazione, l'AR pubblica il numero totale di token emessi (senza rivelarne il contenuto). Questo consente di confrontare il numero di token autorizzati con il numero di schede presenti in bacheca: un eccesso segnala l'introduzione fraudolenta di voti.



2.5 Scrutinio finale

Al termine del periodo di votazione, l'AE pubblica la chiave privata sk_{AE} . Per autenticare questa pubblicazione — impedendo che un avversario pubblichi una sk_{AE} falsa — l'AE produce un annuncio di chiusura firmato con una seconda chiave di lunga durata $sk_{AE\text{-cert}}$, il cui corrispondente certificato pubblico $pk_{AE\text{-cert}}$ è distribuito e verificabile fin dalla fase di setup:

$$\text{ann} = \text{Sign}_{sk_{AE\text{-cert}}}(\text{"chiusura"} \parallel sk_{AE} \parallel h_{\text{root_finale}} \parallel \text{timestamp})$$

La coppia (sk_{AE}, ann) è pubblicata nella Bacheca Pubblica. Chiunque verifica:

$$\text{Vrfy}_{pk_{AE\text{-cert}}}(\text{"chiusura"} \parallel sk_{AE} \parallel h_{\text{root_finale}} \parallel \text{timestamp}, \text{ann}) = 1$$

prima di usare sk_{AE} per decifrare i voti. L'inclusione di $h_{\text{root_finale}}$ nell'annuncio lega la pubblicazione della chiave all'esatto stato finale della bacheca, prevenendo attacchi che sostituiscano sk_{AE} insieme a una bacheca manomessa.

Per ogni scheda c_i , chiunque (incluso l'elettore) può eseguire autonomamente:

$$m_i = \text{OAEP-Dec}(sk_{AE}, c_i) \rightarrow (v_i \parallel id_i)$$

Il conteggio finale è:

$$R = \sum_i v_i$$

Project work

L'AE pubblica in bacheca le coppie (ci, mi). Poiché skAE è ora pubblica, ogni osservatore può verificare in autonomia che $mi = \text{OAEP-Dec}(skAE, ci)$ per ogni scheda: qualsiasi mismatch tra il plaintext pubblicato e quello ottenuto decifrando autonomamente rivela una manipolazione dello scrutinio.

Compromesso esplicito — pubblicazione di skAE: La pubblicazione di skAE al termine consente la verificabilità universale e individuale del risultato, ma fa decadere la segretezza computazionale dei singoli voti: chiunque può decifrare qualsiasi scheda. Poiché però la bacheca contiene solo ciphertext anonimi — non associati ad alcuna identità — e le sessioni TLS verso l'AE erano separate dall'identità dell'elettore, il collegamento tra preferenza e persona rimane spezzato a meno di collusione tra AR e AE. La pubblicazione è autenticata tramite skAE-cert, rendendo rilevabile qualsiasi tentativo di diffondere una chiave falsa. Questo compromesso è discusso in dettaglio in WP3.

2.6 Verifica individuale e universale

2.6.1 Verifica individuale

Fase 1 — Verifica immediata (dopo la trasmissione).

- L'elettore calcola $idc' = \text{SHA-256}(c)$ dal ciphertext che ha prodotto localmente.
- Confronta idc' con la conferma idc ricevuta dall'AE e con il valore pubblicato in bacheca.
- Se $idc' = idc$ e la scheda è visibile in bacheca, la scheda è stata registrata correttamente.
- Facoltativamente, l'elettore verifica la Merkle Proof π per accertarsi che c faccia parte dell'albero firmato: $\text{Vrfy_pkAE}(\text{hroot} \parallel \text{timestamp}, \sigma\text{root}) = 1$.

Fase 2 — Verifica del conteggio (dopo la pubblicazione di skAE).

- L'elettore recupera c dalla bacheca tramite idc .
- Decifra autonomamente: $(v' \parallel id') = \text{OAEP-Dec}(skAE, c)$.
- Confronta v' con la preferenza espressa e id' con l'identificatore id generato localmente prima della cifratura.
- Se entrambi coincidono, il voto è stato incluso e conteggiato correttamente.

2.6.2 Verifica universale

Un osservatore esterno, in possesso di pkAE e pkAE-cert, può:

1. verificare l'annuncio di chiusura: $\text{Vrfy_pkAE-cert}(\text{"chiusura"} \parallel skAE \parallel \text{hroot_finale} \parallel \text{timestamp}, \text{ann}) = 1$;
2. verificare l'integrità della bacheca: $\text{Vrfy_pkAE}(\text{hroot} \parallel \text{timestamp}, \sigma\text{root}) = 1$, dove il messaggio firmato è la concatenazione $\text{hroot} \parallel \text{timestamp}$ (canonicalizzazione fissa, coerente con §2.5.1);
3. con la pubblicazione di skAE, decifrare autonomamente tutte le schede eseguendo $\text{OAEP-Dec}(skAE, ci)$ per ogni ci in bacheca, ricalcolare $R = \sum_i v_i$ e verificare che coincida con il risultato pubblicato; qualsiasi discrepanza tra il plaintext pubblicato dall'AE e quello ottenuto per decrittazione autonoma è prova crittografica di manipolazione dello scrutinio;

Project work

4. verificare che il numero di schede in bacheca corrisponda al numero di token emessi dall'AR.

2.7 Sostituzione del voto prima della chiusura delle urne

Il protocollo consente all'elettore di sostituire il proprio voto prima della chiusura delle urne. La procedura è progettata in modo che la sostituzione non violi il pseudo-anonimato e non consenta all'AE di collegare la scheda originale alla nuova.

Procedura di sostituzione:

1. L'elettore si autentica nuovamente presso l'AR (che verifica che abbia già votato).
2. L'AR emette un secondo token t' , registrando $ht' = \text{SHA-256}(t')$ nel registro dell'AE tramite il canale mTLS (§2.2.4). Contestualmente, l'AR annota internamente che si tratta di un token sostitutivo per quell'elettore e invalida immediatamente il token originale t notificando l'AE che ht deve essere rimosso dal registro delle autorizzazioni valide, indipendentemente dall'esito della revoca della scheda. Questo impedisce che l'elettore possa usare contemporaneamente sia t sia t' . L'AR conosce l'associazione (identità, t , t'), ma l'AE no.
3. L'elettore produce una nuova scheda $(c', t', pk_e', \sigma_v)$ e la trasmette all'AE con la stessa procedura descritta in §2.4.
4. Insieme alla nuova scheda, l'elettore trasmette $idc = \text{SHA-256}(c)$ (la ricevuta del voto originale) e una firma di revoca prodotta con la chiave effimera originale sk_e :

$$\sigma_{inv} = \text{Sign}_{sk_e}(idc \parallel \text{"revocato"})$$

Questa firma prova che il mittente della revoca è lo stesso che aveva firmato la scheda originale.

5. L'AE verifica $\text{Vrfy}_{pk_e}(idc \parallel \text{"revocato"}, \sigma_{inv}) = 1$ (usando pk_e già presente in bacheca), pubblica entrambe le schede e marca c come sostituita con un record:

$$\sigma_{mark} = \text{Sign}_{sk_{AE}}(idc \parallel \text{"sostituita"})$$

6. Al momento dello scrutinio, le schede marcate come sostituite sono escluse dal conteggio.

L'elettore deve aver conservato sk_e dalla fase di trasmissione originale (§2.4.2). La conservazione di sk_e sul dispositivo dell'elettore fino alla chiusura delle urne introduce un rischio: se il dispositivo è compromesso in quel lasso di tempo, un avversario potrebbe produrre una firma di revoca non autorizzata. Questo è un rischio residuo dichiarato, mitigabile con storage locale cifrato.

Rischio di coercizione e leak da σ_{mark} : La possibilità di sostituire il voto riduce la coercizione pre-urne (un coercitore non può essere certo che il voto iniziale rimanga). Tuttavia introduce due rischi. (1) Post-invio: un avversario che osservi l'elettore subito dopo il primo voto potrebbe tentare di forzare una sostituzione. (2) Il record σ_{mark} pubblicato in bacheca è esso stesso un'informazione sensibile: un avversario che monitori il timing delle connessioni TLS verso l'AE durante la fase di revoca potrebbe tentare di correlare la sessione di revoca con l'elettore, riducendo l'anonimato per le schede revocate.

Project work

2.8 Riepilogo dei messaggi scambiati

Flusso principale:

Mittente Destinatario		Contenuto del messaggio
Elettore	AR	Credenziali (canale TLS)
AR	Elettore	Token t (canale TLS)
AR	AE	$ht = \text{SHA-256}(t)$, $\sigma t = \text{Sign_skAR}(ht)$ [canale mTLS AR $\leftrightarrow$ AE, §2.2.4]
Elettore	AE	$M_{\text{voto}} = (c, t, pk_e, \sigma v)$ [nuova sessione TLS]
AE	Elettore	Conferma $idc = \text{SHA-256}(c)$ [stessa sessione TLS]
AE	Bacheca	Coppia (c, idc) , radice Merkle firmata $\sigma root$ [append sulla BP]
AE	Bacheca	Annuncio (sk_{AE}, ann) , coppie (ci, mi) , risultato R [a urne chiuse, sulla BP]
AR	Bacheca	Numero totale token emessi [a urne chiuse, sulla BP]

Flusso di sostituzione del voto (§2.8):

Mittente Destinatario		Contenuto del messaggio
Elettore	AR	Credenziali (nuova autenticazione, canale TLS)
AR	AE	Invalidazione di ht , registrazione di $ht' = \text{SHA-256}(t')$ con $\sigma t' = \text{Sign_skAR}(ht')$ [canale mTLS]
AR	Elettore	Token sostitutivo t' (canale TLS)
Elettore	AE	$M'_{\text{voto}} = (c', t', pk'_e, \sigma v')$ e revoca $= (idc, \sigma inv)$ [nuova sessione TLS]
AE	Bacheca	Nuova scheda (c', idc') , marcatura σ_mark su c , aggiornamento radice Merkle

2.9 Gestione dei casi di errore

Il protocollo descritto nelle sezioni precedenti presenta il flusso principale (*happy path*). Di seguito si descrivono i casi di fallimento più rilevanti e il comportamento atteso del sistema.

Token non valido o già consumato (§2.4.3, verifiche 1–2).

L'AE risponde con un messaggio di errore firmato. Il token è irrecuperabile: l'elettore deve tornare dall'AR per ottenerne uno nuovo, previa nuova autenticazione. L'AR verifica che l'elettore non abbia già un token attivo prima di emetterne un secondo.

Caduta della connessione TLS a metà trasmissione.

Se la sessione cade prima che l'AE risponda con idc , l'elettore non ha conferma. Il token t potrebbe o meno essere stato consumato, a seconda dello stato del server. L'elettore può interrogare la Bacheca Pubblica cercando $\text{SHA-256}(c)$: se idc è presente, il voto è stato registrato; altrimenti il token non è stato consumato e il client deve conservare localmente c , id , sk_e e σv dalla fase di trasmissione originale (§2.4). Se la connessione cade e l'elettore non

Project work

ha ricevuto conferma idc , può interrogare la bacheca cercando $SHA-256(c)$: se presente, il voto è stato registrato e non è necessaria alcuna azione. Se assente, l'elettore può ritrasmettere lo stesso $M_{voto} = (c, t, pk_e, \sigma)$ già costruito, senza rieseguire la cifratura (che è non deterministica e produrrebbe un ciphertext diverso). Qualora i dati locali fossero andati persi, il token t non è riutilizzabile per costruire una nuova scheda valida senza tornare dall'AR. Se il token risulta consumato ma il voto non compare in bacheca — scenario che implica un malfunzionamento o comportamento disonesto dell'AE — l'elettore può segnalarlo alle autorità competenti esibendo il token t . Si noti però che t da solo non costituisce prova crittografica vincolante dell'avvenuta trasmissione: dimostra il possesso del token ma non che un messaggio M_{voto} sia stato ricevuto dall'AE. In assenza di una ricevuta firmata dall'AE, questo scenario rimane un rischio residuo non risolvibile con gli strumenti a disposizione: la segnalazione ha valore procedurale, non crittografico. Questo caso è analizzato come attacco in WP3.

Mancata risposta dell'AE (timeout).

Il client attende una conferma idc entro un timeout configurabile (es. 30 secondi). Allo scadere, esegue la verifica sulla Bacheca Pubblica come descritto sopra, senza ripetere l'invio dello stesso messaggio (per evitare doppio invio).

Firma σ_t non valida nella comunicazione $AR \rightarrow AE$.

Se la firma dell'AR sul token non supera la verifica $Vrfy_pkAR(ht, \sigma_t) = 1$, l'AE rifiuta il token e non lo inserisce nel registro. Questo impedisce a un avversario che intercetti il canale $AR \rightarrow AE$ di iniettare token non autorizzati, anche in presenza di una compromissione parziale del canale di comunicazione.

Nota sulla gestione degli errori: I meccanismi descritti coprono i casi di fallimento più comuni nel flusso di votazione. Scenari più avanzati (es. crash dell'AE durante lo scrutinio, corruzione parziale della bacheca) richiedono misure di recovery che esulano dagli strumenti trattati nel corso e sono menzionati come estensioni future in WP3.

Riepilogo degli algoritmi impiegati:

Algoritmo	Parametri / Variante	Ruolo nel protocollo
RSA-OAEP	Modulo ≥ 2048 bit; sicuro nel ROM	Cifratura del voto (IND-CPA)
RSA-FDH	SHA-256 come hash interno; modulo ≥ 2048 bit	Firma AR su token (ht), firma AE su radice Merkle e annuncio chiusura, firma effimera client su scheda ($c t$)
SHA-256	Output 256 bit	Hash token (ht), identificatori (idc), nodi Merkle; incorporata internamente in RSA-FDH
AES-CTR + MAC (Enc-then-Auth)	$c \leftarrow AES_CTR_kE(m)$, $t \leftarrow Mac_kM(c)$ — chiavi indipendenti derivate tramite HKDF	Authenticated encryption nei canali TLS (CCA-secure)
HKDF	Basata su HMAC-SHA-256	Derivazione di kE e kM dalla chiave di sessione DHE

Project work

Algoritmo	Parametri / Variante	Ruolo nel protocollo
DHE	Diffie-Hellman effimero	Key agreement per ogni sessione TLS
Merkle Tree	SHA-256 per tutti i nodi	Integrità e verificabilità append-only della bacheca
skAE-cert / pkAE-cert	Coppia RSA a lunga durata dell'AE	Autentica la pubblicazione di skAE a urne chiuse

2.10 Considerazioni sull'efficienza

Il protocollo è progettato per restare utilizzabile su dispositivi ordinari, in linea con la scelta del referendum binario.

Lato elettore (costo costante, indipendente da n):

- Generazione della coppia effimera (pk_e, sk_e): un'unica generazione di coppia RSA a 2048 bit, eseguita una sola volta per sessione di voto. Sebbene asintoticamente $O(1)$, questa è l'operazione più costosa lato client: su hardware comune richiede tipicamente 100–500 ms, a seconda dell'implementazione e del sistema operativo. Il costo è accettabile per un'operazione one-shot.
- Cifratura RSA-OAEP (un'esponenziazione modulare con modulo a 2048–3072 bit): $O(1)$, tipicamente nell'ordine dei millisecondi.
- Firma RSA-FDH su $c \parallel t$: $O(1)$.
- Dimensione del messaggio trasmesso: $|N|$ (ciphertext) + $|N|$ (firma) + 256 bit (token) + $|N|$ (pk_e) $\approx$ poche centinaia di byte.

Lato AE (per ogni scheda ricevuta):

- Un confronto hash per validazione del token: $O(1)$.
- Una verifica di firma RSA-FDH: $O(1)$.
- Inserimento di una nuova foglia nel Merkle Tree e aggiornamento della radice: $O(\log n)$ hash.

Verifica (individuale e universale):

- Merkle Proof: $O(\log n)$ hash, con prova di dimensione $O(\log n)$.
- Scrutinio finale: $O(n)$ decrittazioni RSA, eseguibili offline a urne chiuse.

Nel complesso il protocollo non introduce overhead asintotico significativo rispetto a un sistema centralizzato non verificabile, a fronte delle garanzie aggiuntive di integrità, verificabilità pubblica e pseudo-anonimato.

Project work

WP3 – Analisi della Sicurezza

Questo work package analizza la sicurezza del protocollo progettato in WP2 rispetto al modello di minaccia definito in WP1. L'obiettivo è verificare se le proprietà desiderate — autenticità, unicità, segretezza, integrità, verificabilità individuale e universale — siano effettivamente garantite, sotto quali assunzioni di fiducia e con quali limiti residui.

L'analisi segue una struttura per proprietà: per ciascuna si discutono i meccanismi che la realizzano nel protocollo, le ipotesi necessarie alla sua tenuta, i possibili vettori di attacco e il rischio residuo. In chiusura si analizzano i principali compromessi adottati e si verifica l'assenza di miglioramenti ovvi che porterebbero benefici senza perdite in altre proprietà.

3.1 Analisi per proprietà

3.1.1 Autenticità

Solo gli elettori legittimi possono partecipare alla votazione.

Meccanismi a supporto

- L'Autorità di Registrazione (AR) verifica le credenziali dell'elettore rispetto al registro elettorale prima di emettere qualsiasi token.
- Il token t è una stringa casuale a 256 bit: la probabilità che un avversario ne indovini uno valido è trascurabile (2^{-256}).
- Il token è trasmesso esclusivamente su canale TLS, quindi non intercettabile in chiaro da un avversario di rete passivo.
- L'AR comunica all'AE solo l'hash $ht = \text{SHA-256}(t)$, firmato con sk_{AR} . L'AE verifica la firma prima di inserire ht nel registro: token non emessi dall'AR non possono essere iniettati nel sistema.

Attacchi e rischi residui

Per quanto riguarda gli attacchi e i rischi residui relativi all'autenticità, va considerato innanzitutto il rischio di furto di credenziali: se un avversario riesce ad ottenere le credenziali di accesso, quali username e password, di un elettore legittimo, ha la possibilità di autenticarsi al suo posto e di ricevere un token valido. Il protocollo non integra difese crittografiche aggiuntive a protezione di questo aspetto rispetto ai meccanismi propri dell'Autorità di Registrazione, come ad esempio un'autenticazione a due fattori, rendendo questo rischio mitigabile tramite opportune policy ma non eliminabile sul piano puramente crittografico all'interno del sistema descritto.

Un ulteriore fattore di rischio risiede nella possibile compromissione dell'Autorità di Registrazione stessa: un'autorità disonesta o compromessa potrebbe infatti emettere autonomamente token a favore di soggetti privi del diritto di voto. Tale eventualità ricade in un'assunzione di fiducia esplicita del modello, che considera l'Autorità di Registrazione come

Project work

un'entità tipicamente onesta, lasciando dunque questo scenario come rischio residuo dichiarato.

Infine, occorre valutare un eventuale attacco portato al canale di comunicazione tra l'Autorità di Registrazione e l'Autorità Elettorale. Sebbene tale canale risulti protetto tramite autenticazione reciproca mTLS, un avversario che fosse in grado di spoofare l'identità di una delle due autorità potrebbe riuscire a iniettare token arbitrari. Questa circostanza richiederebbe tuttavia la compromissione della CA radice o il possesso delle chiavi private delle autorità coinvolte.

Valutazione: La proprietà è garantita sotto le assunzioni che l'AR sia onesta, che le credenziali degli elettori non siano compromesse e che la CA radice sia affidabile. Il rischio residuo principale è il furto di credenziali, non risolvibile a livello puramente crittografico.

3.1.2 Unicità

Ogni elettore può esprimere al più un voto valido.

Meccanismi a supporto

- L'AR mantiene un registro degli elettori che hanno già ottenuto un token attivo: prima di emetterne uno nuovo, verifica che l'elettore non ne possieda già uno.
- Il token t è monouso: non appena ht viene presentato all'AE durante la votazione, ht è rimosso dal registro delle autorizzazioni. Tentativi di riuso vengono rigettati dall'AE.
- Un replay del messaggio M_{voto} con lo stesso token viene bloccato perché ht è già stato consumato al primo invio valido.

Attacchi e rischi residui

Analizzando i rischi residui legati all'unicità del voto, un primo aspetto riguarda la gestione delle race condition: nell'eventualità in cui due trasmissioni identiche del messaggio contenente il voto raggiungano l'Autorità Elettorale quasi simultaneamente, potrebbe accadere che ciascuna richiesta superi il controllo sulla presenza del token prima che la prima di esse riesca a completare la cancellazione del token stesso. Piuttosto che un limite crittografico, questa problematica riguarda la concorrenza a livello implementativo ed è risolvibile garantendo un locking atomico sul database, imponendo cioè che la cancellazione del token sia definita come un'operazione atomica nel protocollo.

Un'eccezione intenzionale all'unicità è costituita dalla procedura di sostituzione del voto, la cui corretta esecuzione è garantita dall'invalidazione preventiva del token originale prima dell'emissione di quello sostitutivo, oltre che dalla firma di revoca che accerta l'identità del richiedente.

Rimane infine il rischio legato alla sottrazione o furto del token: qualora un avversario riesca ad intercettare la stringa casuale del token — ad esempio compromettendo il dispositivo dell'elettore — avrebbe la possibilità di votare prima del legittimo titolare, il quale troverebbe poi il proprio token già consumato. Tale rischio non appare risolvibile all'interno del sistema

Project work

senza introdurre un legame crittografico (binding) tra il token e l'hardware del dispositivo dell'elettore (come un TPM), strumento che esula dal perimetro considerato.

Valutazione: La proprietà è garantita a livello crittografico (token monouso, firma AR). La gestione della race condition richiede un'implementazione atomica. Il furto del token è il rischio residuo principale.

3.1.3 Segretezza del voto

Nessuna singola entità deve essere in grado di associare il voto all'identità dell'elettore.

Meccanismi a supporto

- Separazione delle conoscenze: l'AR conosce l'identità e t , ma non il voto; l'AE riceve il voto cifrato e ht (non t), ma non conosce l'identità del votante.
- RSA-OAEP garantisce che il ciphertext sia non deterministico (due cifrature dello stesso voto producono ciphertext diversi) e IND-CCA2 sicuro nel Random Oracle Model. L'AE non può dedurre il voto da c prima della chiusura delle urne.
- Le sessioni TLS verso AR e AE sono indipendenti: nessun dato di identità dell'elettore transita verso l'AE.
- La chiave privata sk_{AE} è rivelata solo a urne chiuse, dopo che tutti i voti sono stati raccolti.

Attacchi e rischi residui

Sul fronte della segretezza, il rischio principale e la vera limitazione strutturale del sistema risiedono nella possibile collusione tra l'Autorità di Registrazione e l'Autorità Elettorale. Se tali entità decidessero di collaborare, incrociando i dati sul timing delle connessioni o i valori dei token, potrebbero correlare l'identità dell'elettore con la preferenza espressa. Sebbene il protocollo mitighi tale eventualità attraverso la separazione amministrativa, non è in grado di eliminarla del tutto sul piano crittografico, cosa che richiederebbe l'impiego di tecniche come le firme cieche, attualmente non previste dal perimetro.

A questo si affianca il rischio di correlazione temporale, in quanto un avversario in grado di monitorare il timing delle connessioni TLS verso entrambe le autorità potrebbe cercare di tracciare le sessioni corrispondenti. Questo tipo di attacco viene parzialmente mitigato dall'indipendenza delle sessioni e dalla cifratura dei canali, ma non può essere del tutto scongiurato in contesti caratterizzati da un traffico ridotto o a bassa concorrenza.

Un altro elemento critico è rappresentato dalla pubblicazione della chiave privata dell'Autorità Elettorale al termine delle votazioni. Tale atto fa decadere la segretezza computazionale dei singoli voti, consentendo a chiunque di decifrare i testi cifrati in bacheca. Tuttavia, dal momento che tali testi sono del tutto anonimi e privi di riferimenti identificativi, la pubblicazione della chiave non rivela l'associazione tra persona e voto (a meno della già citata collusione tra le autorità), configurandosi come un compromesso intenzionale e dichiarato.

Project work

Infine, nell'ambito di un referendum binario con risultati particolarmente sbilanciati (ad esempio con il novantanove per cento di voti a favore di una sola opzione), un avversario a conoscenza della partecipazione di un determinato individuo potrebbe dedurre la preferenza su basi statistiche. Si tratta di un rischio intrinseco a qualsiasi sistema elettorale verificabile con esito pubblico, inevitabile se non a costo di rinunciare alla verificabilità universale.

Valutazione: La segretezza è garantita in senso computazionale (IND-CCA2 per i voti, TLS per i canali) sotto l'assunzione critica di non collusione tra AR e AE. Il rischio residuo principale è la collusione, che il protocollo non può eliminare con gli strumenti a disposizione.

3.1.4 Integrità

I voti espressi non devono poter essere alterati, soppressi o introdotti fraudolentemente.

Meccanismi a supporto

- Ogni scheda è firmata dal client con chiave effimera sk_c : qualsiasi modifica al ciphertext c o al token t invalida la firma σ .
- La Bacheca Pubblica è organizzata come Merkle Tree append-only firmato: qualsiasi alterazione, soppressione o riordinamento di schede già inserite invalida la firma $\sigma_{root} = \text{Sign}_{skAE}(\text{hroot} \parallel \text{timestamp})$.
- La radice del Merkle Tree è firmata ad ogni inserimento con timestamp: non è possibile retrodatare o sostituire una versione precedente dell'albero.
- Il numero totale di token emessi dall'AR è pubblicato a urne chiuse: un eccesso di schede rispetto ai token autorizzati segnala l'iniezione fraudolenta di voti.
- La pubblicazione di $skAE$ è autenticata da $skAE\text{-cert}$ (chiave di lunga durata dell'AE): un avversario non può diffondere una $skAE$ falsa senza invalidare l'annuncio.

Attacchi e rischi residui

Un primo rischio residuo è rappresentato dalla soppressione silenziosa delle schede da parte dell'Autorità Elettorale. Il server potrebbe infatti ricevere una scheda del tutto valida ed evitare deliberatamente di inserirla nella bacheca pubblica. L'elettore è in grado di accorgersi di tale omissione verificando l'assenza del proprio identificativo in bacheca, tuttavia, non disponendo di una ricevuta firmata dal server, non possiede una prova crittografica vincolante dell'avvenuta ricezione. Di conseguenza, la contestazione dell'elettore assume un valore meramente procedurale e non dimostrabile dal punto di vista crittografico.

Al contrario, qualsiasi tentativo di alterazione dello scrutinio tramite la pubblicazione di voti in chiaro errati risulterebbe del tutto inefficace e immediatamente rilevabile, poiché qualunque verificatore potrebbe decifrare autonomamente i testi cifrati originali mediante la chiave privata dell'Autorità Elettorale ed evidenziare la manomissione.

In merito all'eventuale iniezione di voti non autorizzati, questa risulta preclusa all'Autorità Elettorale, poiché ogni scheda richiede un token valido preventivamente firmato dall'Autorità

Project work

di Registrazione; inoltre, la corrispondenza tra il numero di schede raccolte e il totale di token emessi offre un ulteriore strumento di controllo pubblico.

Rimane infine la minaccia legata ad una potenziale compromissione della chiave privata dell'Autorità Elettorale prima della chiusura formale delle urne, evento che permetterebbe a un avversario di decifrare i voti man mano che vengono espressi. Tale eventualità viene mitigata mediante procedure di custodia sicura (evitando che la chiave transiti in rete durante la votazione), pur non potendo essere esclusa a livello puramente crittografico senza implementare schemi di decrittazione a soglia.

Valutazione: L'integrità è fortemente garantita per le schede già in bacheca (Merkle Tree firmato). Il rischio residuo principale è la soppressione silenziosa da parte dell'AE, che rimane rilevabile dall'elettore ma non dimostrabile crittograficamente.

3.1.5 Verificabilità individuale

Ogni elettore deve poter verificare autonomamente che il proprio voto sia stato incluso nel conteggio.

Meccanismi a supporto

- L'elettore calcola $idc = \text{SHA-256}(c)$ localmente e confronta con la conferma ricevuta dall'AE e con la bacheca pubblica.
- La Merkle Proof π permette di verificare in $O(\log n)$ che c sia foglia dell'albero firmato, senza dover scaricare tutta la bacheca.
- Dopo la chiusura, l'elettore decifra autonomamente c con sk_{AE} e verifica che v' e id' corrispondano a quanto espresso al momento del voto.

Attacchi e rischi residui

Deve essere considerata la criticità legata alla natura della ricevuta, la quale nel protocollo base non risulta firmata dall'Autorità Elettorale. Nell'eventualità in cui un'autorità disonesta fornisca all'elettore una conferma con un identificativo fasullo, l'utente potrebbe essere indotto a cercare in bacheca un testo cifrato mai trasmesso. La verifica resta comunque solida se l'elettore utilizza l'identificativo calcolato direttamente sul proprio testo cifrato locale, ma la mancanza di firma implica che, qualora la scheda non compaia in bacheca, l'elettore debba condurre una verifica attiva senza potersi affidare esclusivamente al messaggio di conferma.

Un secondo aspetto concerne la vulnerabilità a tentativi di coercizione tramite ricevuta: i meccanismi di verifica individuale (e in particolare la possibilità di decifrare il proprio voto tramite la chiave privata pubblicata a urne chiuse e confrontare l'identificatore) consentono infatti all'elettore di dimostrare a terzi l'orientamento della propria preferenza. Questo fenomeno rappresenta un compromesso intrinseco alla verificabilità individuale, poiché la fornitura di strumenti di riscontro personale aumenta inevitabilmente il rischio di una loro esibizione a fini di coercizione; meccanismi complessi basati su ricevute ingannevoli per contrastare tale problema non sono stati integrati in quanto richiederebbero strumenti crittografici esterni al perimetro stabilito.

Project work

Valutazione: La verificabilità individuale è garantita in modo solido, con due fasi complementari (immediata e post-chiusura). Il compromesso con la resistenza alla coercizione è dichiarato e inerente all'architettura.

3.1.6 Verificabilità universale

Chiunque deve poter verificare la correttezza del risultato finale.

Meccanismi a supporto

- La pubblicazione di skAE a urne chiuse, autenticata da skAE-cert, consente a chiunque di decifrare autonomamente tutte le schede e ricalcolare il risultato R.
- La firma sulla radice del Merkle Tree garantisce che la lista di ciphertext pubblicata non sia stata alterata.
- L'annuncio di chiusura include `hroot_finale`: la pubblicazione di skAE è vincolata allo stato finale della bacheca, impedendo la sostituzione coordinata di skAE e bacheca.
- Il confronto tra schede in bacheca e token emessi dall'AR consente di verificare l'assenza di voti indebitamente aggiunti o rimossi.

Attacchi e rischi residui

Il vettore di attacco principale nella verificabilità universale risiede nella possibile falsificazione dell'annuncio di chiusura. Qualora un avversario riuscisse a compromettere la chiave privata a lunga durata dell'Autorità Elettorale (utilizzata per i certificati), avrebbe la possibilità di pubblicare una chiave privata di decifratura falsa accompagnata da un annuncio apparentemente autentico, inducendo così i verificatori ad applicare una chiave errata nell'analisi dei voti. Tale scenario presuppone tuttavia la compromissione della CA radice, un'ipotesi che rientra tra le assunzioni di fiducia già esplicitamente dichiarate all'interno del modello.

Infine, l'eventuale insorgenza di un'inconsistenza tra i testi in chiaro pubblicati dall'Autorità Elettorale e quelli ottenuti mediante la decifratura autonoma da parte degli osservatori verrebbe rilevata immediatamente, fornendo al contempo una prova crittografica inconfutabile della manipolazione avvenuta durante lo scrutinio.

Valutazione: La verificabilità universale è garantita in modo solido. La pubblicazione di skAE, pur comportando la perdita della segretezza computazionale dei singoli voti, è il compromesso necessario per abilitare la verifica pubblica autonoma.

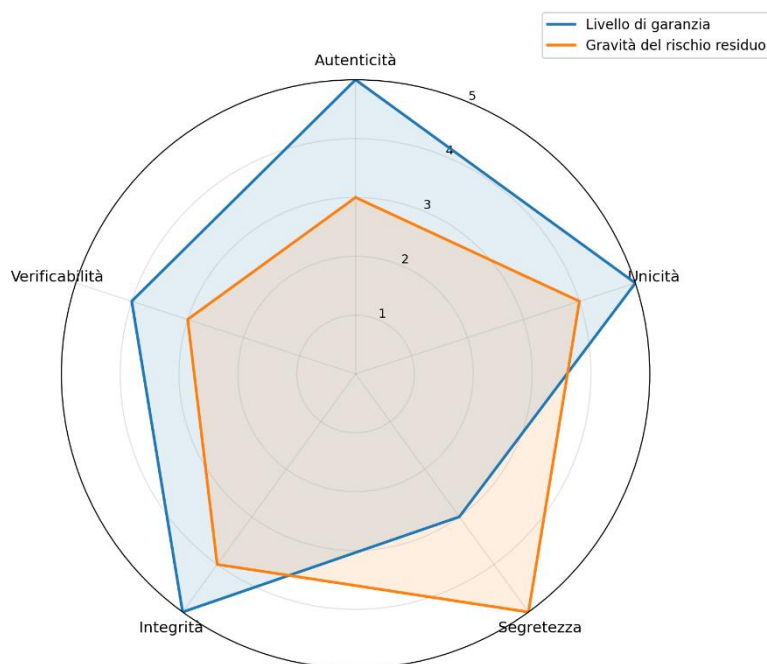
3.2 Riepilogo dei compromessi e analisi critica

La seguente tabella sintetizza le proprietà analizzate, il loro livello di garanzia e le assunzioni necessarie.

Project work

Proprietà	Garanzia	Assunzione critica	Rischio residuo
Autenticità	Forte	AR onesta, CA affidabile	Furto credenziali elettore
Unicità	Forte (crittografica)	Implementazione atomica del token	Furto del token t
Segretezza	Computazionale (IND-CCA2)	Non collusione AR-AE	Correlazione temporale, collusione
Integrità	Forte (Merkle firmato)	AE onesta nella raccolta	Soppressione silenziosa (non dimostrabile)
Verif. individuale	Garantita	Elettore conserva c e id localmente	Possibile sfruttamento per coercizione
Verif. universale	Garantita	CA affidabile per skAE-cert	Compromissione skAE-cert

Lo spider chart che segue assegna un scala da 1-5 al livello di garanzia e alla gravità del rischio residuo a ciascuna proprietà. Questi valori dati sono quantificazioni grafiche e non metriche calcolate formalmente.



3.2.1 Compromesso principale: verificabilità vs. segretezza

La pubblicazione di skAE a urne chiuse è il compromesso più significativo del sistema. Da un lato abilita la verificabilità universale autonoma — chiunque può decifrare le schede e ricalcolare il risultato — dall'altro fa decadere la segretezza computazionale dei singoli voti.

Project work

Poiché i ciphertext sono anonimi, il collegamento tra preferenza e persona rimane spezzato in assenza di collusione AR-AE.

Un'alternativa sarebbe pubblicare solo il risultato R senza $skAE$, affidandosi alla sola firma dell'AE. Ciò preserverebbe la segretezza computazionale, ma eliminerebbe la verificabilità universale: gli osservatori dovrebbero fidarsi dell'AE per la correttezza del conteggio. Il compromesso adottato è più favorevole alla trasparenza, che è una delle priorità dichiarate del progetto.

3.2.2 Limite strutturale: collusione AR-AE

La collusione tra AR e AE è la minaccia strutturalmente più difficile da mitigare con gli strumenti a disposizione. Il protocollo la affronta tramite separazione amministrativa e indipendenza delle sessioni TLS, ma non la elimina crittograficamente.

L'unica soluzione crittograficamente robusta sarebbe l'uso della firma cieca (blind signature): l'AR firmerebbe il token senza vedere t , in modo che nemmeno l'AR potesse collegare il token all'identità dell'elettore una volta emesso. Questa tecnica è menzionata come estensione futura, coerentemente con le linee guida del progetto che escludono strumenti non trattati nel corso dall'implementazione.

3.2.3 Soppressione silenziosa e ricevuta non firmata

Il rischio di soppressione silenziosa da parte dell'AE è rilevabile dall'elettore (che cerca idc in bacheca) ma non dimostrabile crittograficamente in assenza di una ricevuta firmata. Una ricevuta firmata dall'AE — del tipo $Signsk_AE(idc \parallel \text{timestamp})$ restituita all'elettore dopo la registrazione — risolverebbe questo problema. Il costo aggiuntivo sarebbe minimo (una firma RSA per scheda), e non introdurrebbe overhead significativo.

Si tratta di un miglioramento identificabile che non comporta perdite in altre proprietà e che andrebbe adottato in una versione rivista del protocollo. La sua inclusione è raccomandata in sede di implementazione (WP4).

3.2.4 Resistenza alla coercizione

Il protocollo offre una resistenza parziale alla coercizione. La possibilità di sostituire il voto prima della chiusura delle urne (WP2 §2.8) riduce la certezza del coercitore che il voto iniziale rimanga. Tuttavia, il meccanismo di verifica individuale con idc e id può essere sfruttato per dimostrare la preferenza a un terzo dopo la chiusura delle urne (quando $skAE$ è pubblica). Sistemi avanzati come Prêt à Voter o Scantegrity implementano ricevute ingannevoli che permettono all'elettore di “dimostrare” un voto diverso da quello reale, ma richiedono strumenti crittografici fuori perimetro.

3.3 Miglioramenti identificati senza perdita di proprietà

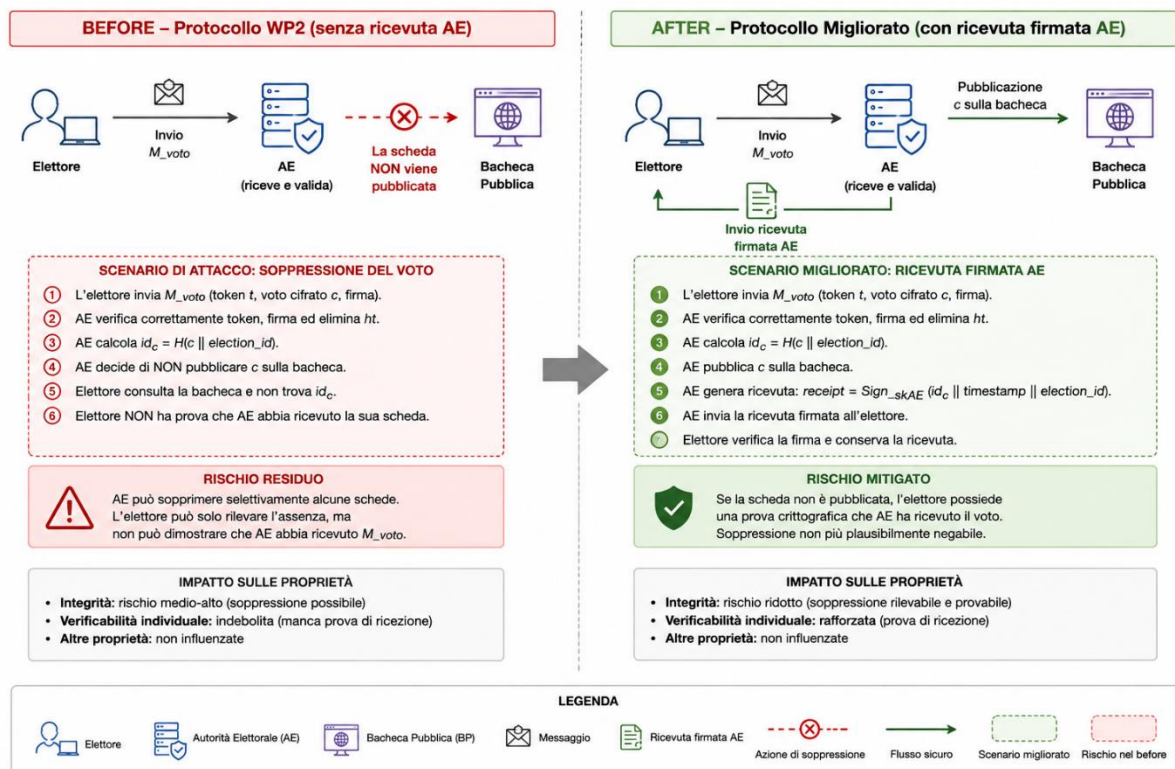
In sede di analisi sono stati identificati i seguenti miglioramenti applicabili al protocollo WP2 senza perdite in nessuna delle proprietà già garantite:

Project work

Miglioramento	Proprietà migliorata	Overhead aggiuntivo
Ricevuta firmata dall'AE (Sign_skAE(idc ts))	Integrità, verif. individuale	1 firma RSA per scheda
Operazione atomica sulla cancellazione di ht	Unicità	Implementativo (nessuno critto.)
Delay randomizzato tra sessione AR e sessione AE	Segretezza (correlazione temporale)	Latenza aggiuntiva per l'elettore
Nonce esplicito nella conferma idc per prevenire replay	Integrità del canale	Minimo (32 bit)

Il miglioramento più significativo è l'introduzione della ricevuta firmata dall'AE, che trasforma il rischio di soppressione silenziosa da rilevabile a dimostrabile crittograficamente. Si raccomanda la sua adozione in WP4.

Questo diagramma spiega più in dettaglio il processo migliorato.



3.4 Conclusioni

Il protocollo progettato in WP2 raggiunge un compromesso ragionevole tra le proprietà desiderate. Le garanzie più forti sono quelle di integrità (Merkle Tree firmato), unicità (token monouso) e verificabilità universale (pubblicazione autenticata di skAE). La segretezza è

Project work

garantita in senso computazionale, con il limite strutturale della collusione AR-AE che non è eliminabile con gli strumenti adottati.

I rischi residui principali sono: (i) la soppressione silenziosa, mitigabile con una ricevuta firmata; (ii) la collusione AR-AE, mitigabile solo con blind signatures (fuori perimetro); (iii) il furto delle credenziali o del token, problemi di sicurezza operativa non risolvibili a livello puramente crittografico.

L'analisi non ha individuato vulnerabilità gravi nel flusso principale del protocollo: i meccanismi crittografici adottati sono corretti, le assunzioni di fiducia sono esplicite e le limitazioni residue sono coerenti con i vincoli dichiarati in WP1 e WP2.

Project work

WP4 – Implementazione e prestazioni

Implementazione simulata del protocollo di e-voting descritto in WP1/WP2/WP3.

Coerenza con il progetto

- Referendum binario: YES/NO.
- Separazione tra Autorità di Registrazione (AR) e Autorità Elettorale (AE).
- Token monouso: t , SHA-256(t) e firma RSA-FDH dell'AR.
- Cifratura del voto con RSA-OAEP.
- Firma della scheda con chiave RSA effimera del client.
- Bacheca pubblica append-only con Merkle Tree SHA-256.
- Firma della root ad ogni inserimento.
- Ricevuta $idc = \text{SHA-256}(c)$ firmata dall'AE: miglioramento raccomandato nel WP3.
- Sostituzione del voto prima della chiusura, con firma di revoca.
- Pubblicazione autenticata di skAE alla chiusura.
- Scrutinio verificabile e verifica individuale/universale.
- Benchmark delle operazioni crittografiche, Merkle Tree, scrutinio e dimensione dei messaggi.

Assunzioni del WP4

Il progetto realizza un ambiente stand-alone simulato. Le entità AR, AE, elettore e bacheca pubblica sono rappresentate da componenti Java locali.

Non viene realizzata una rete reale né una CA/TLS reale: la simulazione riproduce le operazioni crittografiche e il flusso del protocollo definiti nei WP precedenti, permettendo di verificare le proprietà funzionali e misurare i costi computazionali richiesti dal WP4.

Struttura

```
src/main/java/org/example/
├── autorità/
│   ├── ElectionAuthority.java
│   └── RegistrationAuthority.java
├── benchmark/
│   └── PerformanceTest.java
├── bulletin/
│   ├── MerkleProof.java
│   ├── MerkleTree.java
│   └── PublicBulletinBoard.java
├── crypto/
│   ├── HashUtils.java
│   ├── KeyManager.java
│   ├── RSAUtils.java
│   └── SignatureUtils.java
├── entita/
│   ├── Token.java
│   ├── Vote.java
│   ├── VoteRecord.java
│   └── Voter.java
└── protocol/
    └── RegistrationProtocol.java
```

Project work



Requisiti

- Java 17+
- Maven è previsto dal pom.xml, ma il progetto non richiede librerie crittografiche esterne: usa le API Java standard.

Esecuzione

Da e-voting/:

```
javac -encoding UTF-8 -d target/classes $(find src/main/java -name "*.java")  
java -cp target/classes org.example.Main
```

Il programma esegue una simulazione completa e stampa:

- numero di schede pubblicate;
- risultato YES/NO;
- root Merkle finale;
- verifica universale;
- verifica individuale;
- verifica della ricevuta firmata dall'AE.

Benchmark

Per evitare di avviare accidentalmente esperimenti molto lunghi, le dimensioni vengono passate da riga di comando:

```
java -cp target/classes org.example.Main benchmark 10  
java -cp target/classes org.example.Main benchmark 10 100  
java -cp target/classes org.example.Main benchmark 10 100 1000
```

L'output è CSV e comprende:

- generazione chiavi RSA;
- RSA-OAEP encryption;
- RSA-FDH signing;
- RSA-FDH verification;
- inserimento/protocollo + Merkle;
- verifica Merkle;
- scrutinio;
- tempo totale;
- dimensione del messaggio.

La generazione della chiave RSA effimera per ogni voto è intenzionalmente mantenuta nel benchmark perché è prevista dal protocollo WP2; per questo gli esperimenti con migliaia di elettori possono essere costosi.

Project work

Nota implementativa

La simulazione non realizza una rete reale, TLS o una CA: questi elementi sono modellati come separazione logica e uso delle primitive crittografiche nel processo locale. Il WP4 è quindi un ambiente stand-alone, coerente con la traccia, che permette di verificare il flusso e misurare i costi delle operazioni. L'autenticazione delle credenziali è simulata localmente; la gestione sicura delle password non costituisce oggetto del WP4.